



Task 1

Task 2

Challenge 2 description

You want to visit your friend, who lives abroad. It is time to plan the whole journey, both there and back. The trip will be long, so you would like to make it interesting. You do not want to revisit the same places or go along the same paths twice. Also, you do not have much time, so you do not want to head back from any point.

You will represent your planned path by a string containing four letters: 'N' for north, 'S' for south, 'E' for east and 'W' for west. For example, a path going north, east, east, north, west, south would be notated as "NEENWS".

You have already made a plan of the outward part of your journey. How will you plan the shortest path back home, fulfilling the criteria described above?

Write a function:

```
string solution(string &forth);
```

that, given a string forth of length N, which denotes the path leading to your friend, returns one of the shortest possible paths back home and fulfils the above conditions. You can assume that you are heading north at both the beginning and the end of the first part of your journey (the first and the last element in forth are equal to 'N'). Moreover, forth does not contain any occurrence of the letter 'S'.

Examples:

1. Given forth = "NEENWN", your function may return "WWSSE". It may also return "WSWSSE".



USD/INR
+0.10%



Search

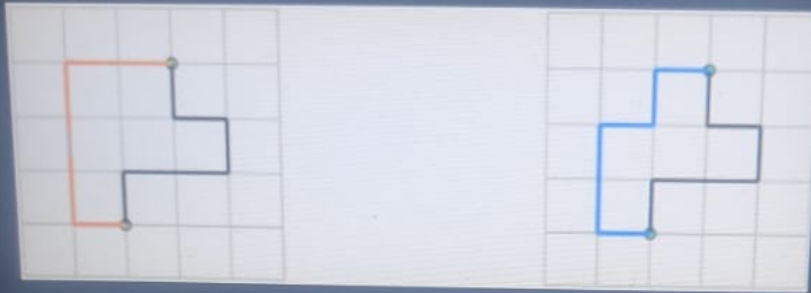


The 2 possibilities are 1. from left side 2. from right side.

Task 1

Task 2

1. Given forth = "NEENWN", your function may return "WWSSSE". It may also return "WSWSSE".



2. Given `forth = "NWNENWNEN"`, your function may return `"ESSSSW"`.



3. Given `forth = "NENENWWWN"`, your function may return `"WSSSSEEE"`.



Sol

2

4

5

6

7

10

11

12

13

14

15

Test output

Compiler:

```
main.o: I
```

`solution`

error: ld



Q Search

10

1

Challenge 2 description

You are given N cards. Each card consists of a suit and a rank. There are four suits, s (Spades), h (Hearts), d (Diamonds) and c (Clubs), and thirteen ranks, ordered (from the lowest to the highest) 2, 3, 4, 5, 6, 7, 8, 9, 10, J (Jack), Q (Queen), K (King) and A (Ace).

Each card is represented by a string in the format " $\langle \text{rank} \rangle \langle \text{suit} \rangle$ ". For example, 10 of Spades is described as "10s", and Queen of Hearts as "QH".

There are six ranked card sets (described in detail below). Sets are ordered by their strength from the weakest to the strongest. Your task is to detect the card sets that can be formed from the given cards. Detecting each card set is scored separately and is worth an equal number of points. If you detect more than one set, **select the strongest one**.

Write a function:

```
Results solution(vector<string> &cards);
```

that, given an array of strings `cards`, returns a `Results` object representing the strongest card set that can be formed.

Assume that the following declarations are given:

```
struct Results { string set_name; vector<string>  
selected_cards; };
```

- `set_name` is a string which should be the name of detected set (see below);
- `selected_cards` is a list of strings, which should be made of cards that form the detected set. Cards can be listed in any order.



Search



Esc



F1



F2



F3



F4



F5

`selected_cards; };`

- `set_name` is a string which should be the name of detected set (see below);
- `selected_cards` is a list of strings, which should be made of cards that form the detected set. Cards can be listed in any order.

Scoring:

- detecting each card set is scored separately and is worth an equal number of points;
- in tests worth 50% of points only the strongest set's `set_name` is checked. Checking `selected_cards` field of the Result object is omitted in these tests.

Card sets:

There are six card sets ordered by their strength from the weakest (single card) to the strongest (a triple and a pair).

Set 1: single card

- **Name:** "single card"
- **Description:** A single card of the highest rank; the suit does not matter.

For example, for `cards = ["2H", "4H", "7C", "9D", "10D", "KS"]`, your function should return:

```
{ "set_name" = "single card", "selected_cards" = ["KS"] }
```

Set 2: pair



Search



Esc



F1



F2



F3



F4



F5

Set 2: pair

- **Name:** "pair"
- **Description:** Two cards sharing the same rank; suits do not matter. If there are multiple pairs, return any one with the highest rank.

For example, for cards = ["AS", "10H", "8H", "10S", "8D"], one of the correct results is:

```
{ "set_name" = "pair", "selected_cards" = ["10H", "10S"] }
```

Set 3: triple

- **Name:** "triple"
- **Description:** Three cards sharing the same rank; suits do not matter. If there are multiple ways to choose a triple, return any with the highest rank.

For example, for cards = ["AS", "JS", "AH", "AD", "10D", "AC"], one of the correct results is:

```
{ "set_name" = "triple", "selected_cards" = ["AH", "AD", "AC"] }
```

Set 4: five in a row

- **Name:** "five in a row"
- **Description:** Five cards of consecutive ranks starting with the highest possible rank; suits do not matter. If there are multiple ways to choose five such cards, return any.

For example, for cards = ["6H", "7S", "8S", "9S", "10S", "JD", "JC", "KC", "AC"], one of the correct results is:



Search



Esc



F1



F2



F3



F4



F5

to choose five such cards, return any.

For example, for cards = ["6H", "7S", "8S", "9S", "10S", "JD", "JC", "KC", "AC"], one of the correct results is:

```
{ "set_name" = "five in a row", "selected_cards" = ["7S", "8S", "9S", "10S", "JC"] }
```

Set 5: suit

- **Name:** "suit"
- **Description:** Five cards sharing the same suit; the ranks do not matter. If there are multiple ways to choose five cards with the same suit, choose any set with the highest suit. The order of the suits (from the highest to the lowest) is S, H, D, C.

For example, for cards = ["2D", "4D", "6D", "8D", "9D", "AC", "KC", "QC", "JC", "7C"], one of the correct results is:

```
{ "set_name" = "suit", "selected_cards" = ["2D", "4D", "6D", "8D", "9D"] }
```

Set 6: a triple and a pair

- **Name:** "a triple and a pair"
- **Description:** Five cards, consisting of a triple (three cards of the same rank) and a pair (two cards of the same rank). If there are multiple ways to choose this set, choose one with the highest rank of the triple, then one with the highest rank of the pair. The suits do not matter.

For example, for cards = ["10D", "10H", "10C", "2S", "2H", "2D", "JH", "JC"], one of the correct results is:



Search



Esc



F1



F2



F3



F4



F5


```
{ "set_name" = "suit", "selected_cards" = ["2D", "4D", "6D", "8D", "9D"] }
```

Set 6: a triple and a pair

- Name: "a triple and a pair"
- Description: Five cards, consisting of a triple (three cards of the same rank) and a pair (two cards of the same rank). If there are multiple ways to choose this set, choose one with the highest rank of the triple, then one with the highest rank of the pair. The suits do not matter.

For example, for cards = ["10D", "10H", "10C", "2S", "2H", "2D", "3H", "3C"], one of the correct results is:

```
{ "set_name" = "a triple and a pair", "selected_cards" = ["10D", "10H", "10C", "3H", "3C"] }
```

Constraints:

Assume that:

- N is an integer within the range [1..10];
- the elements of cards are all distinct;
- each string in array cards is a correct representation of a card in "<rank><suit>" format.

In your solution, focus on correctness. The performance of your solution will not be focus of the assessment.

Copyright 2009–2024 by Codility Limited. All Rights Reserved. Unauthorized copying, publication or disclosure prohibited.



Search



Esc



F1



F2



F3



F4



F5



Task 1

Task 2

For the first number, the greater digit number should be in string "s", and the smaller digit number in "t", for the remaining numbers the smaller digit number in "s" and greater one in string "t".

Challenge 2 description

In this task you are given two strings of digits that represent (possibly very large) integers. Your goal is to make those numbers as close to one another as possible. In other words, you want to minimize the absolute value of their difference. You can swap some of the corresponding digits (e.g. the first digit of the first number with the first digit of the second number, the second digit of the first number with the second digit of the second number, etc.). Swapping the digits is an extremely tiring task, so you want to make as few swaps as possible.

Write a function:

```
int solution(string &S, string &T);
```

that, given two strings S and T, both of length N, returns the minimum number of swaps needed to minimize the difference between the two numbers represented by the input strings.

For example, given S="29162" and T="10524" your function should return 2. We can swap the second and the fourth digits and obtain "20122" and "19564". The difference between the numbers is 558 and the number of swaps is 2. One can easily check that the difference is the smallest possible. Note that we could obtain the same difference by swapping the first, third and fifth digits, but this solution requires three swaps.

Write an **efficient** algorithm for the following assumptions:

- lengths of S and T are equal and within the range [1..100,000];
- S and T consist only of digits and no other characters;
- neither S nor T contain leading zeroes.

Copyright 2009–2024 by Codility Limited. All Rights Reserved. Unauthorized copying, publication or disclosure prohibited.



34°C
Haze



Search

esc



F1



F2



F3



F4



F5

Challenge 1 description

leetcode 1466

There is a network with N roads and $N + 1$ cities. The cities are labeled with distinct integers within the range $[0..N]$.

Roads connect the cities in such a way that there is exactly one way to travel between any two of the cities. In other words, the network forms a *tree*.

The roads in the network are too narrow to accommodate two cars. For this reason, every road (that connects cities A and B) is oriented in one of two possible ways: either from A to B , or from B to A . If the road is oriented from A to B , then every car traveling from B to A has to give way to cars traveling from A to B . This, naturally, makes traveling from A to B much faster than traveling from B to A .

A big hospital was recently founded in the city labeled 0 . For that reason, the citizens have decided to rearrange the orientation of the roads so that everyone can get to the hospital as quickly as possible. This means that the trip from every city to the 0 th city should not go through any road that faces against the current direction of travel. In order to minimize the cost of this operation, they would like to reorient as few roads as possible.

Write a function:

```
int solution(int A[], int B[], int N);
```

that, given the description of the network as two arrays A , B of N integers each, returns the minimum number of roads that must be reoriented in order to make everyone's trip to the hospital as fast as possible. Arrays A and B describe the network in the following way: for each K in the range $[0..N-1]$, there is a road between cities labeled $A[K]$ and $B[K]$ that is oriented from $A[K]$ to $B[K]$.

Examples:



96°F
Haze



Search



Challenge 1 description

There are N tiles (numbered from 0 to $N-1$). Each tile is made of two squares that are colored either red (represented by the letter "R") or green (represented by "G"). A tile is described by a two-character string representing the respective colors of the left and right squares. The tiles cannot be rotated (which means that "RG" and "GR" tiles are different). Two tiles can be placed next to each other if the color of their adjacent squares is the same.

What is the length of the longest possible sequence that can be created using the provided tiles?

Write a function:

```
int solution(char *A[], int N);
```

that, given an array A of N strings, representing the tiles, returns the maximum number of tiles that can be arranged in a sequence.

Examples:

1. Given $A = ["RR", "GR", "RG", "GR", "GR", "RR"]$,



we can select tiles 0, 2, 3, 4, 5 (underlined in the picture above) and arrange them into the sequence GR - RR - RG - GR - RR:



34°C
Haze



Search

esc



F1



F2



F3



F4



F5

!

@

#

\$



Task 1

Task 2

Examples:

1. Given $A = ["RR", "GR", "RG", "GR", "GR", "RR"]$,



we can select tiles 0, 2, 3, 4, 5 (underlined in the picture above) and arrange them into the sequence GR - RR - RG - GR - RR:

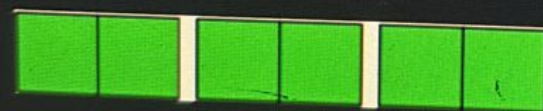


The function should return 5.

2. Given $A = ["GG", "GG", "RR", "GG", "RR"]$,



we can select tiles 0, 1, 3 (underlined in the picture above) and arrange them into the sequence GG - GG - GG:



The function should return 3.

3. Given $A = ["RG", "GR", "RG", "GR"]$,



34°C
Haze



Search

The function should return 5.

2. Given $A = ["GG", "GG", "RR", "GG", "RR"]$,



we can select tiles 0, 1, 3 (underlined in the picture above) and arrange them into the sequence GG - GG - GG:



The function should return 3.

3. Given $A = ["RG", "GR", "RG", "GR"]$,



all tiles can be used without reordering them. The function should return 4.

4. Given $A = ["RG", "RG", "RG"]$, the function should return 1.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- each string in array A is one of the following: "RR", "RG", "GR", "GG".

Copyright 2009–2024 by Codility Limited. All Rights Reserved. Unauthorized copying, publication or disclosure prohibited.



34°C
Haze



Search

esc



F1



F2



F3



F4



F5

1

2

3

\$

4 ₹

You are given an array A consisting of N integers (N is divisible by 3) and an integer K . In one move, you can increase or decrease any element of A by 1. The task is to modify array A using such moves in order to maximize the difference between the $(N/3)$ th largest element and the $(N/3)$ th smallest element of the resulting array. You can perform **at most** K moves.

For example, array $A = [9, 12, 15, 5, 7, 13]$ consists of $N = 6$ elements, so the $(N/3)$ th (that is, the 2nd) largest element is 13 and the $(N/3)$ th (2nd) smallest element is 7.

Write a function:

```
int solution(vector<int> &A, int K);
```

that, given an array A and an integer K , returns the maximum possible difference between the $(N/3)$ th largest element and the $(N/3)$ th smallest element of A after performing at most K moves, as described above.

Example:

1. Given $A = [8, 8, 8, 7, 7, 7, 7, 7, 7, 7, -8, -8]$ and $K = 1$, the function should return 1. We can increase one 7 once. The resulting array is $[8, 8, 8, 8, 7, 7, 7, 7, 7, 7, -8, -8]$. The 4th biggest number is 8 and the 4th smallest is 7.

2. Given $A = [-5, 1, 1, 4, 4, 4, 7, 4, 6]$ and $K = 6$, the function should return 7. We can increase one 4 three times, increase the 6 once and decrease both 1s once. The resulting array is $[-5, 0, 0, 4, 4, 7, 7, 4, 7]$. The 3rd biggest number is 7 and the 3rd smallest is 0.

3. Given $A = [-7, -6, -3, -2, -2, -2, -2, -2, -2, -2, -2, -1]$ and $K = 5$, the function should return 3. We can decrease one -2 three times and decrease the -3 twice. The resulting array is $[-7, -6, -5, -5, -2, -2, -2, -2, -2, -2, -2, -1]$. The 4th biggest number is -2 and the 4th smallest is -5 .

4. Given $A = [6, 6, 6, 6, 6, 6]$ and $K = 3$, the function should return 1. We can decrease two 6s once. The resulting array is $[5, 5, 6, 6, 6, 6]$. The 2nd biggest number is 6 and the 2nd smallest is 5.

Write an **efficient** algorithm for the following assumptions:

- N is an integer within the range $[3..150,000]$;
- N is divisible by 3;
- K is an integer within the range $[0..500,000,000]$;
- each element of array A is an integer within the range $[-300,000,000..300,000,000]$.